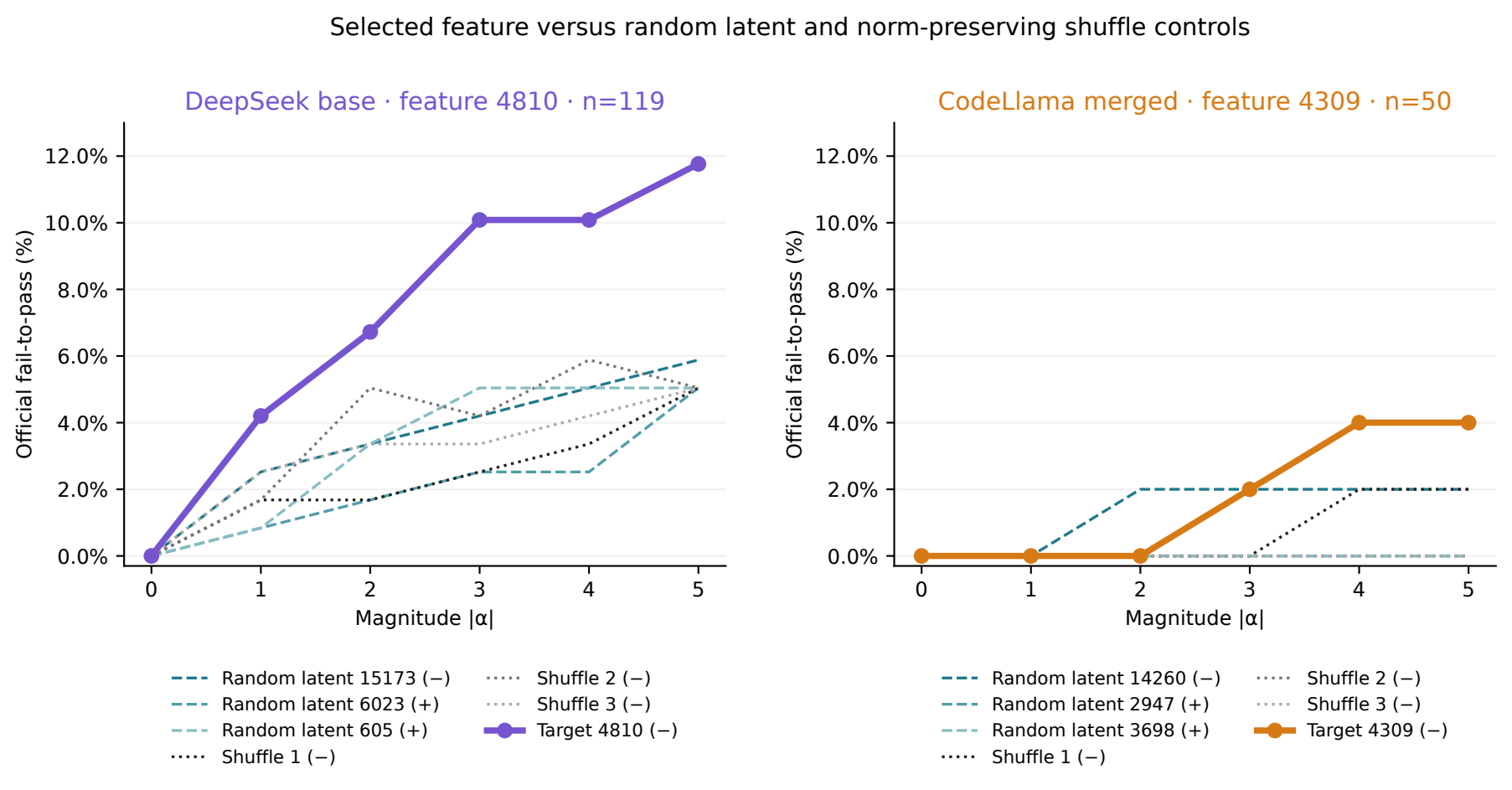
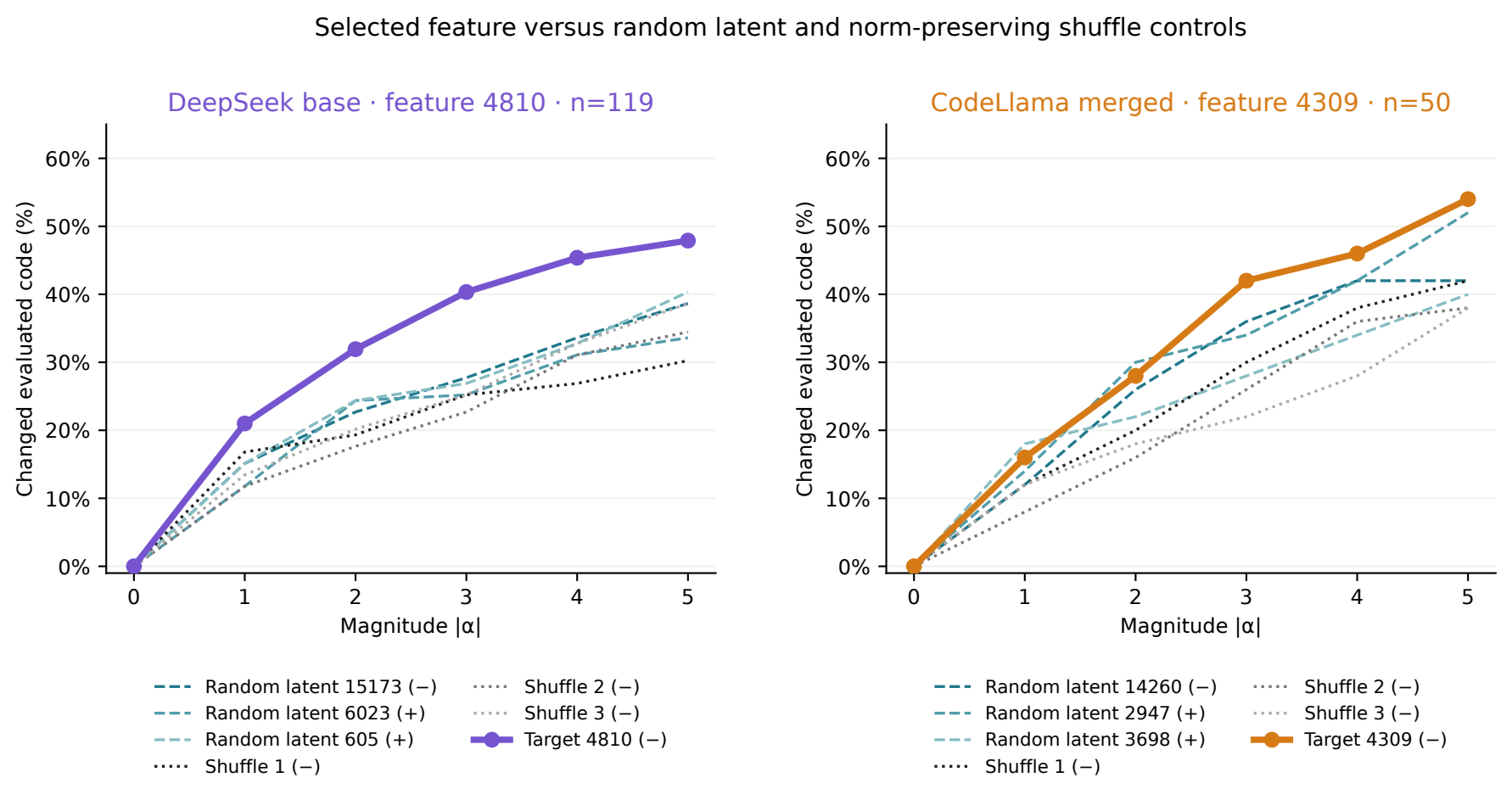
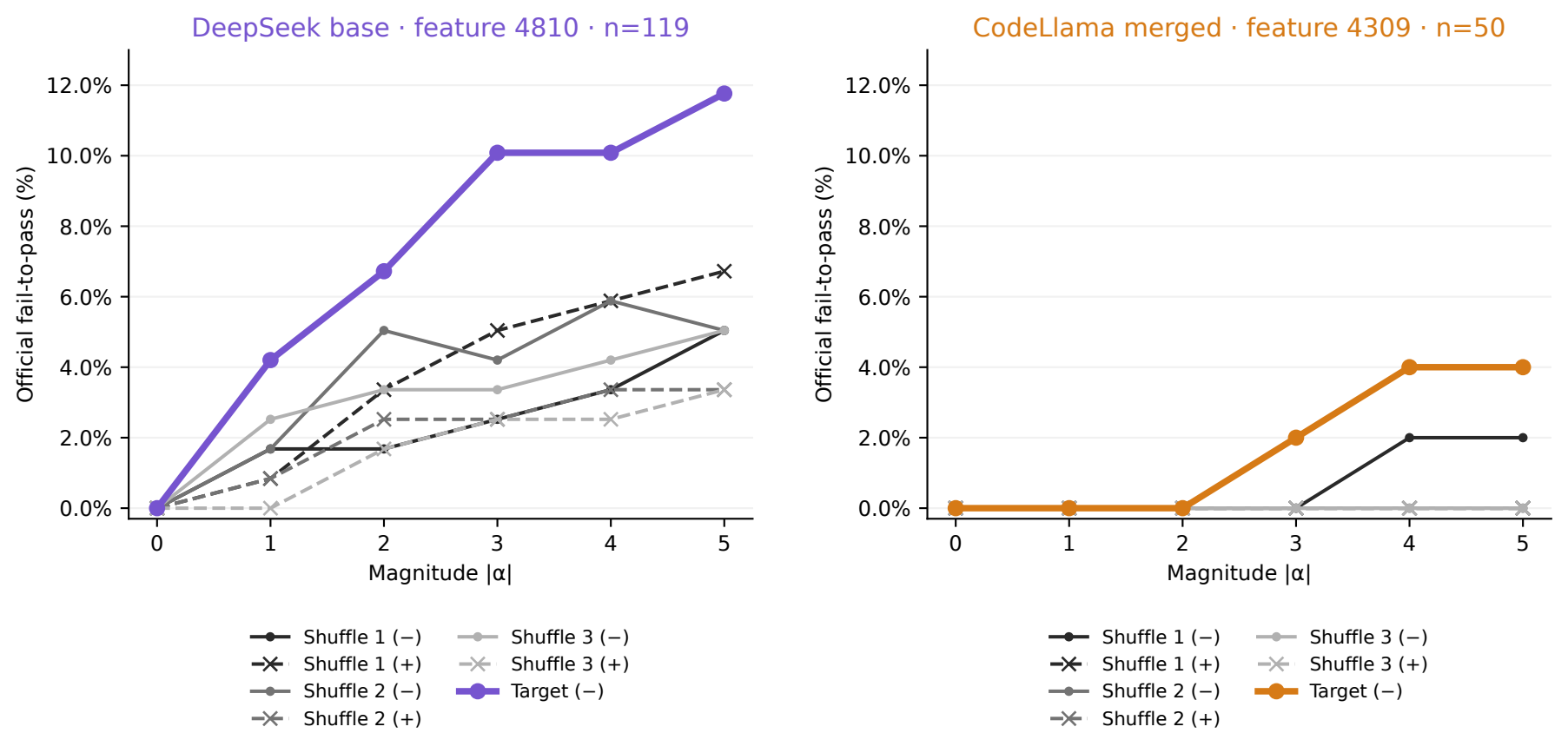
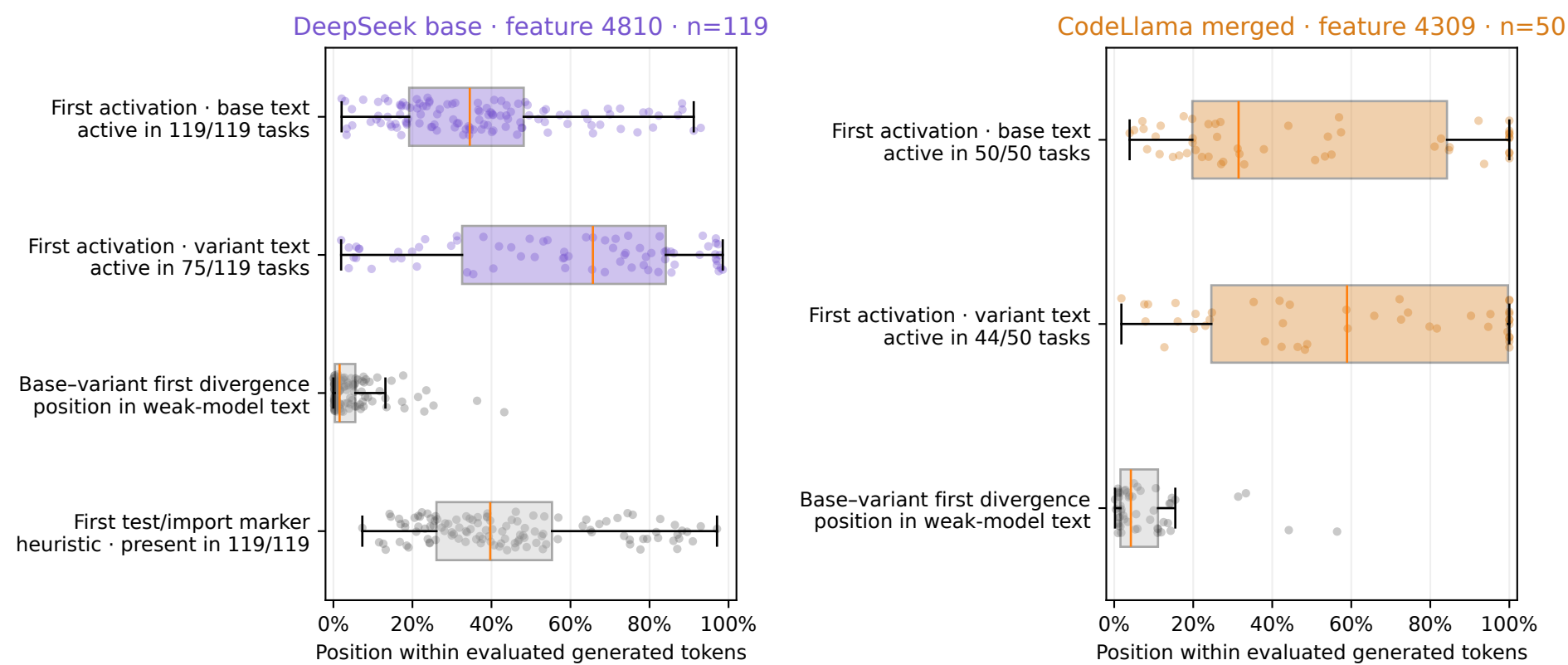
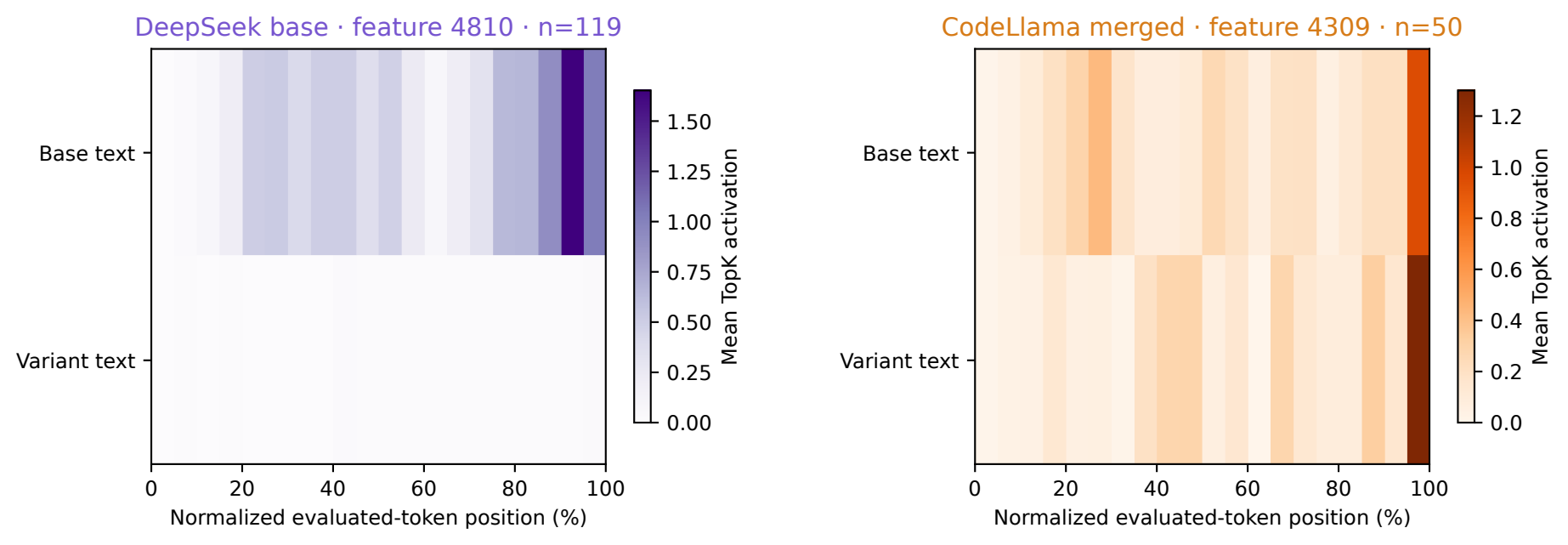
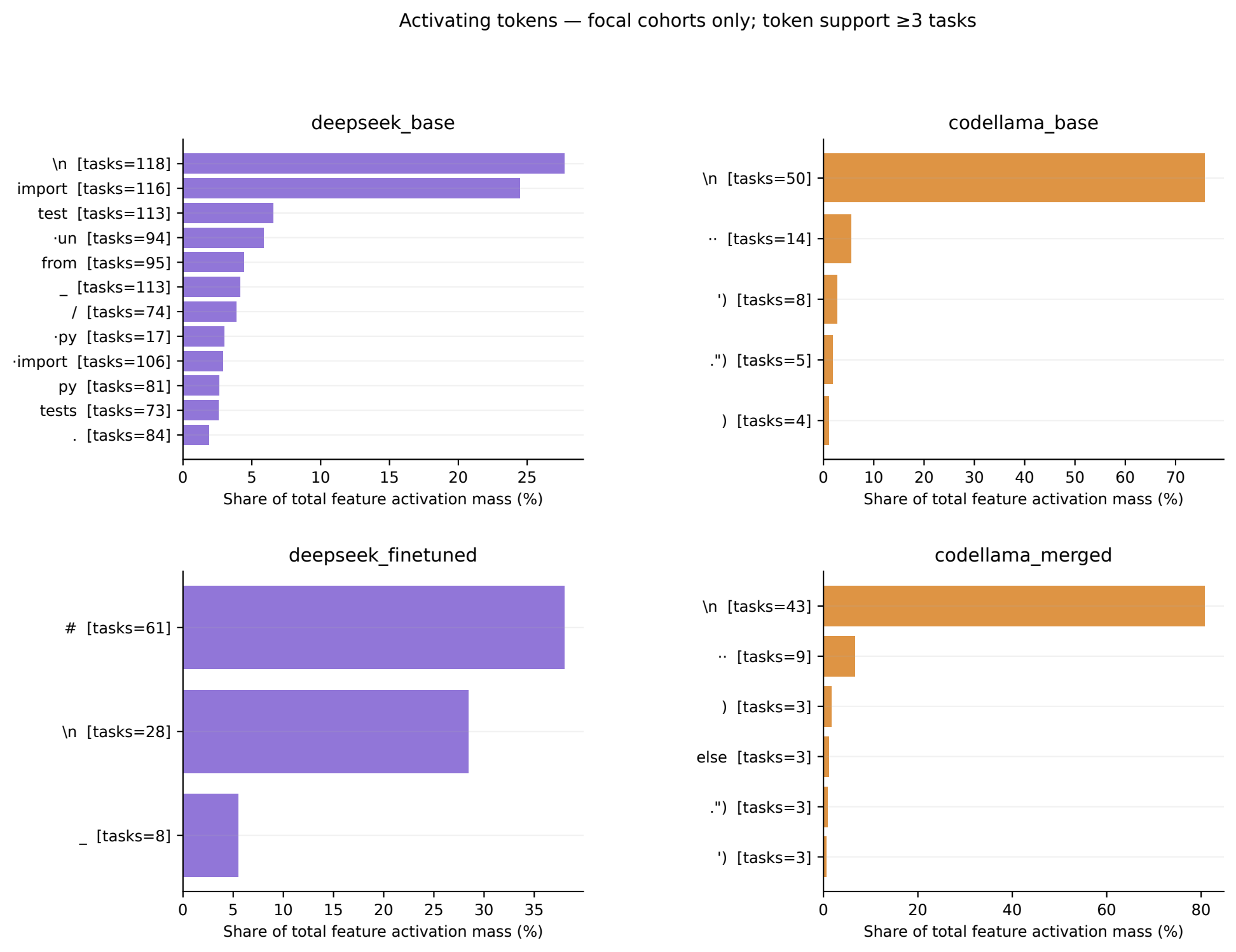

Selected feature versus random latent and norm-preserving shuffle controls
DeepSeek base · feature 4810 · n=119
CodeLlama merged · feature 4309 · n=50
Official fail-to-pass (%)
12.0%
10.0%
8.0%
6.0%
4.0%
2.0%
0.0%
0
1
2
3
4
5
Magnitude |α|
Random latent 15173 (−)
Random latent 6023 (+)
Random latent 605 (+)
Shuffle 1 (−)
Shuffle 2 (−)
Shuffle 3 (−)
Target 4810 (−)
Random latent 14260 (−)
Random latent 2947 (+)
Random latent 3698 (+)
Shuffle 1 (−)
Shuffle 2 (−)
Shuffle 3 (−)
Target 4309 (−)

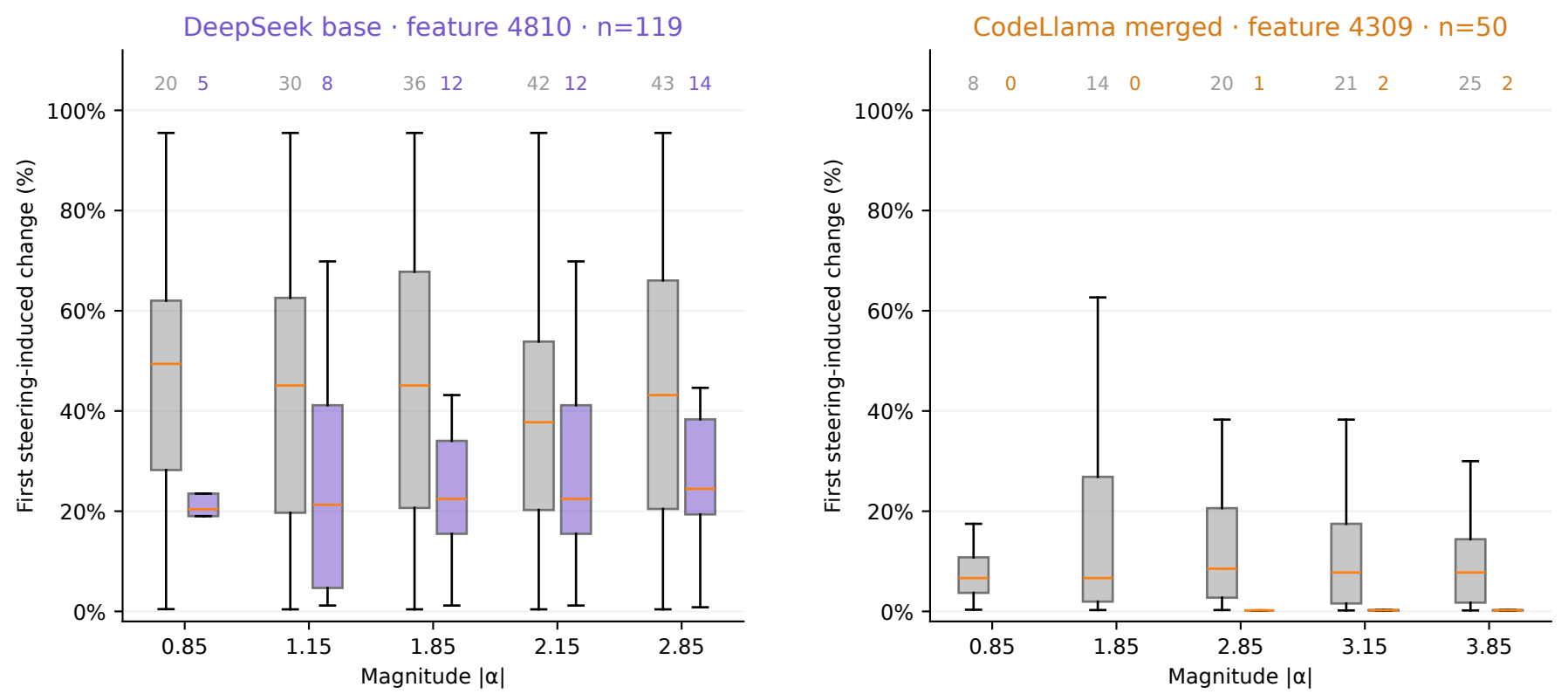

Selected feature versus random latent and norm-preserving shuffle controls
DeepSeek base · feature 4810 · n=119
CodeLlama merged · feature 4309 · n=50
Changed evaluated code (%)
Magnitude |α|
0%
10%
20%
30%
40%
50%
60%
0
1
2
3
4
5
Random latent 15173 (−)
Random latent 6023 (+)
Random latent 605 (+)
Shuffle 1 (−)
Shuffle 2 (−)
Shuffle 3 (−)
Target 4810 (−)
Random latent 14260 (−)
Random latent 2947 (+)
Random latent 3698 (+)
Shuffle 1 (−)
Shuffle 2 (−)
Shuffle 3 (−)
Target 4309 (−)


DeepSeek base · feature 4810 · n=119
CodeLlama merged · feature 4309 · n=50
Official fail-to-pass (%)
12.0%
10.0%
8.0%
6.0%
4.0%
2.0%
0.0%
0
1
2
3
4
5
Magnitude |α|
Shuffle 1 (−)
Shuffle 1 (+)
Shuffle 2 (−)
Shuffle 2 (+)
Shuffle 3 (−)
Shuffle 3 (+)
Target (−)


DeepSeek base · feature 4810 · n=119
First activation · base text active in 119/119 tasks
First activation · variant text active in 75/119 tasks
Base–variant first divergence position in weak-model text
First test/import marker heuristic · present in 119/119
0%
20%
40%
60%
80%
100%
Position within evaluated generated tokens
CodeLlama merged · feature 4309 · n=50
First activation · base text active in 50/50 tasks
First activation · variant text active in 44/50 tasks
Base–variant first divergence position in weak-model text
0%
20%
40%
60%
80%
100%
Position within evaluated generated tokens


DeepSeek base · feature 4810 · n=119
Base text
Variant text
0
20
40
60
80
100
Normalized evaluated-token position (%)
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Mean TopK activation
CodeLlama merged · feature 4309 · n=50
Base text
Variant text
0
20
40
60
80
100
Normalized evaluated-token position (%)
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Mean TopK activation


Activating tokens — focal cohorts only; token support ≥3 tasks
deepseek_base
\n [tasks=118]
import [tasks=116]
test [tasks=113]
·un [tasks=94]
from [tasks=95]
_ [tasks=113]
/ [tasks=74]
·py [tasks=17]
·import [tasks=106]
py [tasks=81]
tests [tasks=73]
. [tasks=84]
0 5 10 15 20 25
Share of total feature activation mass (%)
codellama_base
\n [tasks=50]
·· [tasks=14]
') [tasks=8]
.") [tasks=5]
) [tasks=4]
0 10 20 30 40 50 60 70
Share of total feature activation mass (%)
deepseek_finetuned
[tasks=61]
\n [tasks=28]
_ [tasks=8]
0 5 10 15 20 25 30 35
Share of total feature activation mass (%)
codellama_merged
\n [tasks=43]
·· [tasks=9]
) [tasks=3]
else [tasks=3]
.") [tasks=3]
') [tasks=3]
0 20 40 60 80
Share of total feature activation mass (%)



Colored: corrected · gray: changed but still failing · numbers above: tasks · unchanged outputs omitted, not placed at 100%

Semantic case studies — corrected programs and changed-but-failing programs

DEEPSEEK · BigCodeBench/937 · α=−5

```
@@ -29,16 +29,5 @@
     return char_counts

-#test_task.py
-import unittest
-from task import task_func
-
-class TestTask(unittest.TestCase):
-    def test_task_func(self):
-        # Test case 1: Basic test
-        input_str = "Hello, World!"
-        expected_output = {'l': 3, 'o': 2, 'h': 1, 'e': 1, 'w': 1,
'r': 1, 'd': 1}
-        self.assertEqual(task_func(input_str), expected_output)
-
-        # Test case 2: Empty string
-        input_str = ""
-        expected_output = {}
+# Test the function
+input_str = "Hello, World!"
+print(task_func(input_str))
```

PASS: removed failing test-module import; solution body unchanged.
Baseline first activation: 40.5% · first code change: 40.5%

DEEPSEEK · BigCodeBench/475 · α=−5

```
@@ -67,5 +67,5 @@

     # Create a histogram of the data
-    ax =
data['dates'].dt.month.value_counts().sort_index().plot(kind='bar')
+    ax =
data['dates'].dt.month.value_counts().sort_index().plot(kind='bar',
color='blue', alpha=0.5)

     # Set the title and labels
@@ -77,7 +77,2 @@

     return ax
-
-#tests/test_task_func.py
-import unittest
-import pandas as pd
-from task_func import task_func
```

FAIL: test imports removed, but evaluator still reports missing locale.
Baseline first activation: 93.0% · first code change: 77.8%

CODELLAMA · BigCodeBench/490 · α=−5

```
@@ -26,7 +26,10 @@
     'Emma'
     """
-    return xmltodict.parse(s)
+    result = xmltodict.parse(s)
+    with open(file_path, 'w') as f:
+        json.dump(result, f)
+    return result

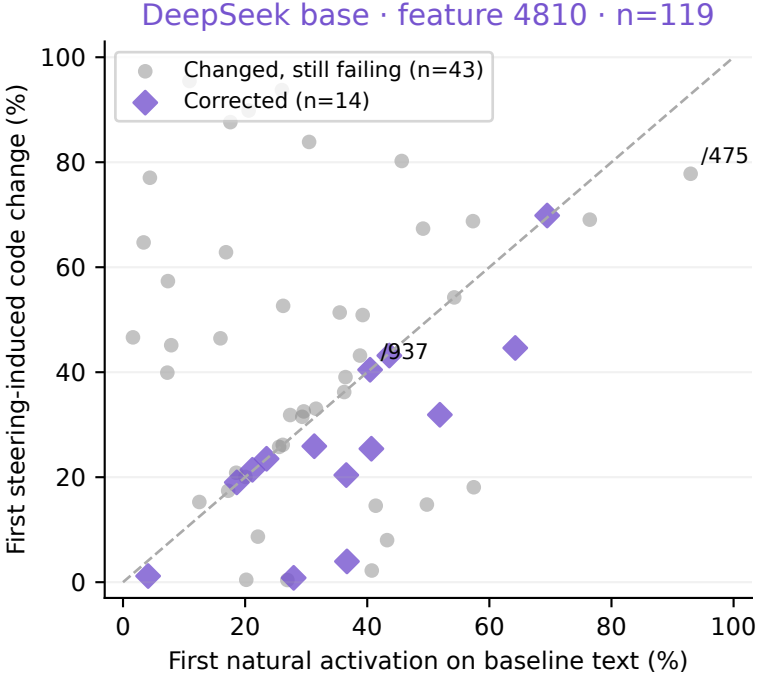
-def task_func2(s, file_path):
[Additional duplicate task_func_2 omitted from this excerpt.]
```

PASS: file-writing behavior restored; duplicate helper remains in output.
Baseline first activation: 2.0% · first code change: 0.2%

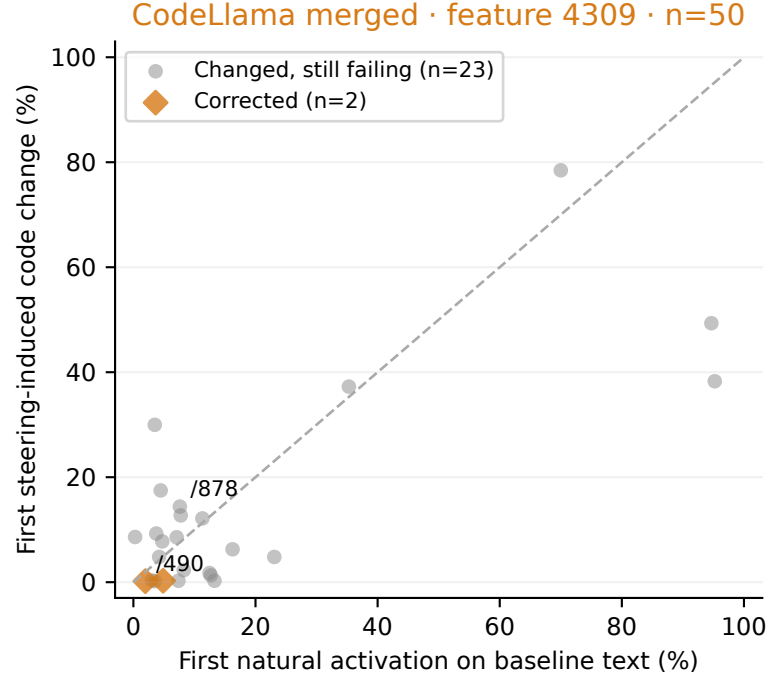
CODELLAMA · BigCodeBench/878 · α=−5

```
@@ -56,30 +56,22 @@
     if target is None or target not in data:
         raise ValueError("The target column name is not in the
DataFrame.")
+    if test_size is None or test_size < 0 or test_size > 1:
+        raise ValueError("The fractional size of the test set is
not between 0 and 1.")
     if random_state is None:
-        random_state = None
-    if test_size is None:
-        test_size = 0.2
-    if test_size > 1:
-        raise ValueError("The fractional size of the test set is
greater than 1.")
-    if test_size < 0:
-        raise ValueError("The fractional size of the test set is
less than 0.")
-    if test_size < 0.01:
-        raise ValueError("The fractional size of the test set is
less than 0.01.")
-    if test_size > 1:
-        raise ValueError("The fractional size of the test set is
greater than 1.")
-    if test_size < 0:
[Excerpt ends here; complete diff in CASE_INSPECTION.md]
```

FAIL: validation changes, but repeated checks remain and the task still fails.
Baseline first activation: 7.6% · first code change: 14.4%



Changed outputs: naturally inactive=0; unaligned active anchor=0



Changed outputs: naturally inactive=1; unaligned active anchor=1

α=−5 · Below the diagonal: code changed before the feature first activated naturally. Steering here is unconditional.